5Stone팀 기획제안서

프로젝트명: 대화 로그 기반 운영지식 생성기

팀원1: 하장한
팀원2: 강희원
팀원3: 강준현
팀원4: 배성연

목차

1. 프로젝트 개요 3
1.1 문제정의 및 솔루션 3
1.2 주 타겟 및 Stakeholder 3
1.3 핵심 가치 3
2. 기존 사례 분석 분석 및 사업화 가능성 3
2.1 유사 제품/서비스 3
2.2 사업화 가능성 3
2.2.1 비즈니스 모델 정의 3
2.2.2 시장규모 분석 3
2.2.3 상용화 시 예상 사업규모 3
3. User Story Mapping 4

3.1 페르소나 정의 4
3.1.1 승객 페르소나: 직장인 마이클 (35세) 4
3.1.2 운전자 페르소나: 제니퍼 (42세) 4
3.2 사용자 여정 (User Journey) 5
3.2.1 승객 마이클의 사용자 여정 5
3.2.2 운전자 제니퍼의 사용자 여정: 저녁 여유 시간 ❹버 운행 5
3.3 Epics & User Stories 6
3.3.1 Epic 1: 승객의 차량 요청 및 매칭 6
3.3.2 Epic 2: 운전자의 승객 요청 수락 및 픽업 6
3.3.3 Epic 3: 운행 완료 및 결제 프로세스 7
4. 설계 및 구현 7
4.1 기술 분석 7
4.1.1 기술 스택 7
4.1.2 개발 및 협업 환경 7
4.1.3 핵심 기술 및 알고리듬 (Optional) 7
4.1.4 하드웨어 구축 기술 (Optional) 8
4.2 시스템 설계 8
4.2.1 시스템 개요 8
4.2.2 소프트웨어 아키텍처 8
4.2.3 데이터 설계 8
4.2.4 API 정의 8
5. 과제 수행 계획 9
5.1 일정계획 9
5.2 업무분장 9
5.3 코드 품질 향상을 위한 협업방식 9
5.4 성과 측정 방안 9

1. 프로젝트 개요

기존의 고객지원 자동화 시스템은 고객의 요청을 처리하기 필요한 문의 유형(intent), 필요한 정보(slot), 적용 정책(policy), 위험 조건(risk), 처리 흐름(workflow)과 같은 정보를 사람이 직접 설계해야 했기 때문에 도입 비용이 높고 운영 효율이 떨어지는 문제가 있었다. 그러나 실제 서비스 환경에는 이미 방대한 상담 로그가 축적되어 있으며, 이 안에는 반복되는 문의 패턴, 추가 질문 방식, 정책 적용 기준, 상담사 이관 조건 등 중요한 운영 지식이 포함되어 있다. 다만 이러한 지식은 비정형 대화 형태로 흩어져 있어 구조화되지 않았고, 그 결과 재사용이나 자동화에 활용하기 어려운 상태로 남아 있었다.

본 프로젝트는 이러한 비효율을 제거하기 위해 비정형 상담 데이터를 분석하여 실행 가능한 지식 형태로 변환하여 일련의 스키마를 자동으로 설계하는 새로운 패러다임을 제시한다. 과거 상담 로그로부터 머신러닝을 활용해 의도와 워크플로우 초안을 자동으로 도출함으로써 도메인 온보딩 비용을 크게 절감하고, 실시간 상담 과정에서는 대화의 상태를 지속적으로 추적하며 정책과 리스크 조건을 적용해 응답의 정확성과 안전성을 동시에 확보한다. 또한 위험도가 높은 상황에서는 단순 이관이 아닌 판단 근거와 요약 정보가 포함된 형태로 상담사에게 전달하여 의사결정을 지원한다.

결과적으로 이 시스템은 기존의 상담사 중심(Customer Support) 구조를 '에이전트 우선(Agent-First)' 구조로 전환하고, 상담 과정에서 발생하는 모든 예외와 사례를 다시 지식으로 축적하여 지속적으로 성능이 향상되는 운영 환경을 만든다. 이를 통해 자동 응답과 상담사 보조가 유기적으로 결합된 데이터 기반 AICC 운영 및 실행 플랫폼을 구현하는 것을 목표로 한다.

1.1 문제정의 및 솔루션

[problem]

- 방대한 상담 로그가 있음에도 인텐트 분류와 워크플로우 설계를 사람이 직접 수행해야 하며, 이 과정에 수개월의 시간과 높은 비용이 소요됨
- 비정형 상담 데이터가 존재하지만 이를 실행 가능한 규칙이나 시나리오로 변환하는 과정이 수작업에 의존함
- 정책 변경이나 신규 서비스 출시 시, 이를 문서화하고 시스템에 반영하는 과정이 수작업에 의존해 대응 속도가 느림
- 생성형 AI 기반 응답은 비즈니스 정책과 리스크 조건을 안정적으로 준수하기 어려워 환각 및 오응대 위험이 존재함
- 챗봇 서비스에서 상담사 이관 시 대화 맥락과 판단 근거가 충분히 전달되지 않아 처리 효율과 응대

품질이 저하됨

[solution]

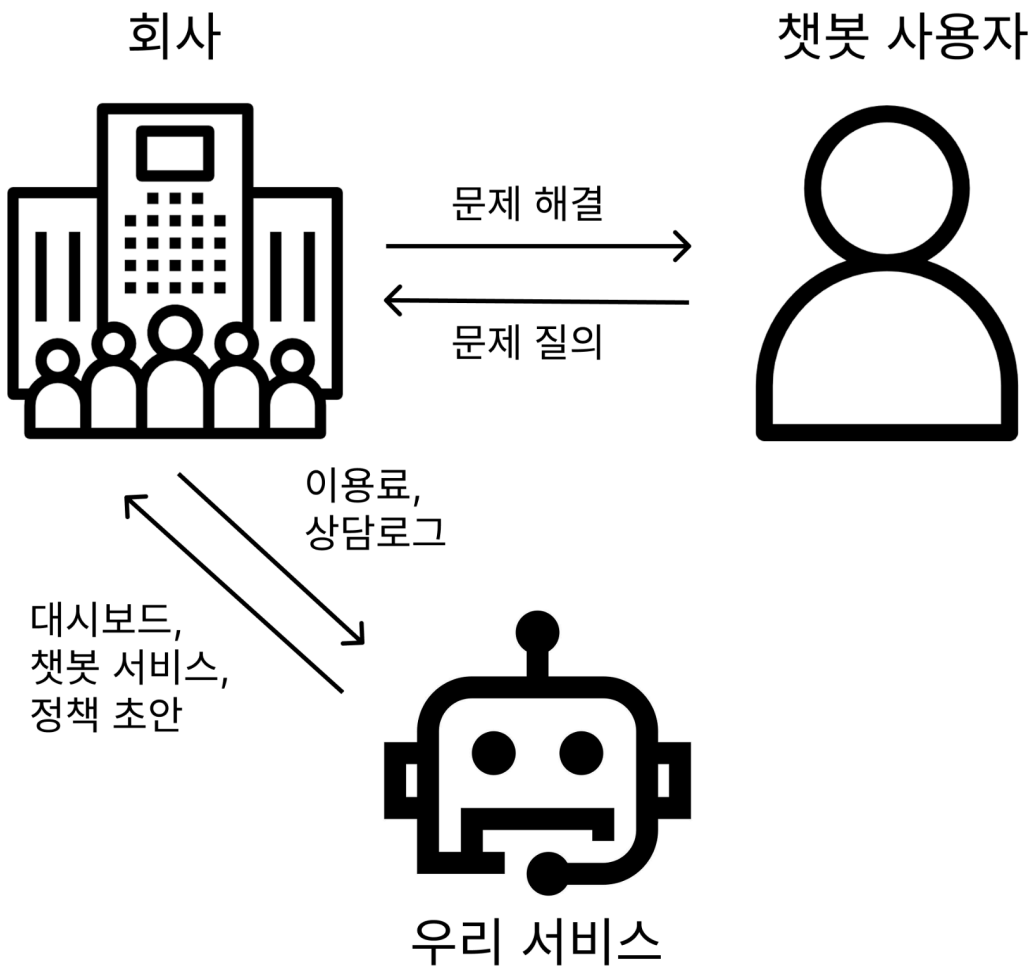
- 상담 로그를 기반으로 인텐트와 워크플로우를 자동 추출하여 수작업 설계 의존도를 최소화하고 구축 시간을 획기적으로 단축
- 비정형 상담 데이터를 분석해 Intent, Slot, Workflow 형태의 실행 가능한 구조로 변환하여 재사용 가능한 지식으로 축적
- 정책 문서 변경이나 신규 서비스 업데이트 시, 관련 로직과 워크플로우를 자동으로 재생성·반영하여 운영 반영 속도 개선
- 정책, 리스크 조건을 에이전트 실행 단계에 내장하고 상태 기반 실행 엔진을 활용해 응답이 사전 정의된 규칙과 안전 기준을 준수하도록 제어
- 고위험 또는 불확실한 케이스는 상담사에게 이관하되, 요약·추천 답변·판단 근거를 함께 제공하여 의사결정 효율과 응대 품질 향상

1.2 주 타겟 및 Stakeholder

메인타겟

- 큰 규모의 CS팀 없이 챗봇 상담 서비스를 병목없이 운용하고 싶은 기업
- CS팀을 운용할 여력이 부족한 자영업자

Stakeholder



1.3 핵심 가치

핵심가치	상세내용
자동 설계 전환	상담 로그를 기반으로 인텐트와 워크플로우를 자동 추출하여 수작업 설계 의존도를 제거하고 구축 시간을 획기적으로 단축

데이터 기반 구조화	비정형 상담 데이터를 분석해 Intent, Slot, Workflow 형태의 실행 가능한 구조로 변환하여 재사용 가능한 지식으로 축적
정책 반영 자동화	정책 문서 변경이나 신규 서비스 업데이트 시, 관련 로직과 워크플로우를 자동으로 재생성·반영하여 운영 반영 속도 개선
거버넌스 내재화	정책, 리스크 조건을 에이전트 실행 단계에 내장하고 상태 기반 실행 엔진을 활용해 응답이 사전 정의된 규칙과 안전 기준을 준수하도록 제어
Human-in-the-Loop 협업	고위험 또는 불확실한 케이스는 상담사에게 이관하되, 요약·추천 답변·판단 근거를 함께 제공하여 의사결정 효율과 응대 품질 향상

2. 기존 사례 분석 및 사업화 가능성

2.1 유사 제품/서비스

Channel.io

서비스 개요

고객 대화를 상담뿐 아니라 CRM·마케팅·매출로 연결하며, 모든 고객 접점을 하나의 메시징 기반 플랫폼으로 통합함

차별화 포인트

채널톡은 다양한 채널과 기능을 통합하는 데 강점이 있지만, 운영 정책은 여전히 사람이 설계해야 함. 우리는 상담 로그 기반으로 실행 가능한 정책과 워크플로우를 자동 생성하는 데 초점을 둠

2.2 사업화 가능성

본 프로젝트는 2026년 에이전틱 AI와 성과 기반 과금 체계가 보편화된 시장 환경에서 명확한 사업적 타당성을 보유한다. 현재 기업용 AI 시장은 단순 질의응답을 넘어 AI가 스스로 목표를 설정하고 실행하는 행동 중심의 에이전트 단계로 진입했으며, 가트너 등 주요 기관의 예측대로 서비스 계약의 절반 이상이 실제 문제 해결 건수에 따라 비용을 지불하는 성과 기반 모델로 재편되고 있다. 이러한 흐름 속에서 본 플랫폼은 기존의 경직된 룰 기반 시스템과 신뢰성이 낮은 단순 LLM 챗봇의 간극을 메우는 독보적인 포지셔닝을 취한다.

기술적 핵심인 '로그 기반 워크플로우 역설계'는 수개월이 소요되던 지식 공학 과정을 획기적으로 단축하여 초기 온보딩 비용을 50% 이상 절감하는 경제적 우위를 제공한다. 또한, 규제가 엄격한 금융이나 공공 섹터에서 가장 우려하는 할루시네이션 리스크를 결정론적 상태 머신과 리스크 게이트 구조로 차단함으로써 운영 안정성을 확보한다. 이는 단순한 기술적 차별화를 넘어, AI 도입의 가장 큰 장벽인 '신뢰도' 문제를 해결하여 실질적인 시장 침투력을 높이는 핵심 동인이 된다.

수익 모델은 서비스 안정성을 보장하는 기본 구독료에 AI가 독립적으로 상담을 종결한 건당 수수료를 합산하는 하이브리드 방식을 채택하여 고객사의 비용 효율성과 플랫폼의 수익성을 동시에 추구한다. 초기 진입 전략으로는 노후화된 레거시 시스템을 보유한 금융 및 통신사를 대상으로 실제 로그 분석을 통한 단기 PoC(Proof of Concept)를 전개하여 기술적 실효성을 즉각적으로 증명하는 방식을 취한다. 데이터 보안 리스크에 대해서는 온프레미스 배포 옵션과 고도화된 개인정보 마스킹 기술로 대응하며, 복잡한 예외 상황에서는 인간 상담사로의 정교한 이관 체계를 유지함으로써 시스템의 완결성을 높인다. 결과적으로 본 프로젝트는 데이터 자산화를 통한 비용 절감과 실행 안전성이라는 양면의 가치를 통해 엔터프라이즈 AI 시장에서의 강력한 사업적 경쟁력을 확보한다.

2.2.1 비즈니스 모델 정의

본 프로젝트의 비즈니스 모델은 비정형 상담 데이터를 기업의 핵심 지식 자산으로 전환하는 '지식 추출'과 이를 안전하게 실행하는 '거버넌스 엔진'을 결합한 B2B SaaS 기반의 하이브리드 수익 구조를 지향한다. 단순히 채팅 UI를 제공하는 기존의 소통 도구 수준을 넘어, 고객사의 운영 비용을 직접적으로 절감하고 상담 해결률이라는 실질적인 비즈니스 성과를 창출하는 데 핵심 가치를 둔다.

수익 구조는 서비스의 지속성을 담보하는 기본 구독료, AI의 성능과 직결된 성과 기반 과금, 그리고 초기 도입 시 발생하는 데이터 분석 서비스료로 삼원화된다. 기본 구독료는 워크스페이스 관리 및 도메인 팩 유지보수 인프라 비용을 충당하며, 성과 기반 과금은 상담사 개입 없이 AI가 독자적으로 상담을 완결한

건당 수수료를 부과함으로써 고객사의 비용 절감액 일부를 수익으로 환수하는 구조다. 또한, 대규모 상담 로그를 파싱하여 워크플로우 초안을 생성하는 과정에서 발생하는 데이터 처리 비용은 별도의 분석료로 책정하여 초기 수익성을 확보한다.

가치 제안 측면에서 본 플랫폼은 '구축의 효율성'과 '실행의 안전성'이라는 양면적 우위를 점한다. 기존 엔터프라이즈 솔루션들이 수개월에 걸쳐 수동으로 시나리오를 설계해야 했던 것과 달리, ML 기반의 자동 유도 알고리즘을 통해 구축 기간을 50% 이상 단축하여 고객사의 시장 진입 속도를 가속한다. 동시에 규제 산업군에서 요구하는 엄격한 정책 준수와 리스크 통제를 위해 리스크 게이트와 결정론적 상태 머신을 적용함으로써, 생성형 AI의 불안정성을 배제한 신뢰 기반의 상담 자동화를 실현한다.

시장 진입 및 확장 전략은 'PoC 주도의 레거시 전환'을 필두로 한다. 노후화된 룰 기반 챗봇을 운용 중인 금융, 공기업, 통신사를 1차 타겟으로 설정하여 실제 상담 로그를 활용한 단기 기술 검증을 통해 솔루션의 실효성을 즉각적으로 입증한다. 이후 확보된 도메인별 레퍼런스를 바탕으로 보안과 권한 관리가 필수적인 중견·대기업 시장으로 확장하며, 최종적으로는 상담사 코파일럿 기능을 통해 인간 상담사와 AI가 유기적으로 협업하는 통합 운영 플랫폼으로 자리매김하여 록인(Lock-in) 효과를 창출한다.

2.2.2 시장규모 분석

본 프로젝트가 타겟으로 하는 시장은 2026년 현재 '단순 챗봇'을 넘어선 '에이전틱 AICC(AI Contact Center)' 및 '엔터프라이즈 AI 에이전트' 시장으로, 국내외적으로 폭발적인 성장세를 기록하고 있다. 글로벌 컨설팅 그룹과 시장조사기관의 자료를 종합할 때, 전 세계 대화형 AI 시장은 연평균 20% 이상의 고성장을 지속하여 2026년 기준 약 300억 달러 규모에 이를 것으로 전망되며, 특히 한국 시장 역시 인건비 상승과 디지털 전환(DX) 가속화에 따라 수조 원대 규모의 AICC 시장이 형성되어 있다.

본 프로젝트의 전체 도달 가능 시장(TAM)은 국내외 컨텍 센터 솔루션 및 기업용 AI 소프트웨어 시장 전체를 포함한다. 특히 기존의 상담원 보조 도구와 단순 시나리오형 챗봇 시장이 AI 에이전트 기반의 자율형 상담 시스템으로 빠르게 대체되고 있다는 점이 핵심적인 시장 동력이다. 여기서 본 플랫폼이 실질적으로 공략 가능한 유효 시장(SAM)은 보안과 정책 준수가 필수적인 금융, 공기업, 통신 산업군 및 대규모 상담 데이터를 보유한 중견·대기업 섹터다. 국내 금융권(은행, 보험, 증권) 약 150여 개사와 주요 공기업 및 통신사, 그리고 고도화된 CS가 필요한 상위 5,000여 개의 이커머스 기업을 포함할 때, 국내에서만 연간 약 5,000억 원 이상의 서비스 가능 시장이 존재한다고 판단된다.

수익 획득 시장(SOM) 측면에서는 초기 3년 내에 노후화된 레거시 챗봇을 운용 중인 금융 및 공공기관 50개소와 중대형 이커머스 고객사 500개 확보를 1차 목표로 설정한다. 금융권의 경우 연간 상담 구축 및 유지보수 예산이 수십억 원 단위에 달하며, 이커머스 기업들 또한 상담 해결률 제고를 위해 월 수백만 원 규모의 SaaS 비용을 지출할 의사가 명확하다. 본 프로젝트의 핵심 경쟁력인 '로그 기반 역설계'를 통해

구축 비용을 50% 절감하고, 상담 해결률에 따른 성과 기반 과금 모델을 적용할 경우, 기업당 평균 객단가(ARPU)를 기존 솔루션 대비 1.5배 이상 높이면서도 고객사의 총소유비용(TCO)은 낮추는 시장 침투가 가능하다.

결론적으로, 2026년의 시장 환경은 단순 기술 도입 단계를 지나 '실질적인 운영 효율'과 '비용 절감'을 정량적으로 증명해야 하는 성숙기에 접어들었다. 본 플랫폼은 데이터 자산화를 통한 구축 기간 단축과 리스크 관리라는 명확한 가치를 제안함으로써, 고비용 수작업에 의존하던 기존 컨설팅 중심의 구축 시장과 신뢰도가 낮은 범용 AI 봇 시장 사이의 거대한 공백을 빠르게 점유할 수 있는 충분한 시장 규모와 성장 잠재력을 확보하고 있다.

2.2.3 상용화 시 예상 사업규모

본 프로젝트 상용화 시 예상되는 사업 규모는 2026년 기준 '성과 기반 과금(Outcome-based Pricing)' 모델의 정착과 '에이전틱 AI' 도입 가속화에 따라 비약적인 성장이 가능할 것으로 판단된다. 현재 국내 AICC(AI 컨택센터) 시장은 연간 수조 원 규모로 형성되어 있으며, 그중에서도 본 솔루션이 타겟팅하는 고정밀 워크플로우 제어 및 자동 구축 영역은 연평균 30% 이상의 고성장이 예상되는 핵심 세그먼트다.

구체적인 매출 규모는 엔터프라이즈형 도입과 SaaS형 확산이라는 투트랙 전략을 통해 산출된다. 금융 및 공공기관 등 대형 고객사의 경우, 기존의 수동 시나리오 구축 및 운영 비용을 50% 이상 절감하는 가치 제안을 통해 연간 고객당 평균 매출(ARPU)을 약 1억 원 내외로 설정할 수 있다. 초기 3년 내 타겟 산업군 내 50여 개의 선도 기업을 확보할 경우, 엔터프라이즈 부문에서만 연간 50억 원 이상의 안정적인 반복 매출(ARR) 발생이 기대된다.

중대형 이커머스 및 일반 기업 대상의 SaaS 부문은 상담 해결 건당 수수료를 부과하는 성과 기반 모델을 적용하여 확장성을 극대화한다. 월 평균 상담 건수 1만 건 이상의 고객사 500개를 확보하고, AI의 상담 자율 해결률(Resolution Rate)을 70% 수준으로 유지할 경우, 건당 500~1,000원의 수수료 산정 시 연간 약 35억~70억 원 규모의 추가 매출 확보가 가능하다. 이는 기존의 단순 계정당 과금 방식보다 고객사의 비용 절감 효과를 투명하게 증명함과 동시에 플랫폼의 수익성을 극대화하는 구조다.

결과적으로 본 프로젝트는 상용화 안착 시 초기 3~5년 내에 연 매출 100억 원 이상의 비즈니스 규모를 달성할 수 있는 충분한 시장 잠재력을 보유하고 있다. 특히 데이터 기반의 자동화 구축(Induction) 기술은 글로벌 진출 시에도 현지 데이터에 즉각 대응할 수 있는 확장성을 지니므로, 일본 및 북미 시장 진출을 고려할 경우 사업 규모는 현재 추정치의 5배에서 10배 이상으로 확대될 가능성이 높다. 이는 단순 솔루션 판매를 넘어 상담 지식의 자산화를 주도하는 '지능형 운영 플랫폼'으로서의 독점적 시장 지위를 구축하는 과정이 될 것이다.

3. User Story Mapping

3.1 페르소나 정의

1) Persona 1 - 고객지원 운영 책임자

고객지원 체계를 설계하고 운영 방향을 정한다.

페인 포인트

- 반복적으로 들어오는 문의를 상담사가 직접 처리하느라 CS운영 효율이 떨어진다.
- 문의 유형, 처리 기준, 예외 대응 흐름이 체계적으로 정리되지 않아 상담 품질 편차가 발생한다.
- 정책이나 운영 기준이 바뀔 때 이를 빠르게 반영하고 전체 상담 흐름에 연결하기 어렵다.

기대하는 가치

- 문의 유형과 처리 흐름의 초안을 빠르게 확보해 운영 설계에 드는 시간을 줄이고 싶다.
- 반복 문의는 자동화하고 상담사는 더 복잡하고 중요한 상담에 집중할 수 있기를 바란다.
- 운영 효율과 함께 응대 품질의 일관성도 높일 수 있는 체계를 기대한다.

2) Persona 2 - 현업 정책 담당자(Service Handler)

고객지원 상담사로부터 이관된 사례를 검토하고, 실제 정책·규정·운영 기준에 따라 최종 처리 여부를 판단한다.

페인 포인트

- 상담사로부터 전달받은 내용이 불완전하거나 맥락이 부족해 다시 확인해야 하는 경우가 많다.
- 유사 사례, 최신 정책, 예외 처리 기준이 한눈에 정리되어 있지 않아 판단에 시간이 걸린다.
- 반복적으로 비슷한 케이스를 처리하지만, 그 과정이 체계적으로 축적되지 않아 업무 효율이 떨어진다.

기대하는 가치

- 이관 시 필요한 정보와 고객 맥락이 한 번에 정리되어 빠르게 판단할 수 있기를 기대한다.
- 관련 정책, 유사 사례, 권장 처리 방향이 함께 제시되어 일관되고 정확한 결정을 내리길 원한다.
- 반복되는 판단과 처리 기준이 누적·구조화되어, 시간이 지날수록 업무 부담이 줄어드는 것을 기대한다.

3) Persona 3 - 최종 고객

실제로 고객센터를 이용하는 사용자

페인 포인트

- 자동 응답이 내 상황을 정확히 이해하지 못하거나, 해결이 어려운 경우에도 적절한 다음 단계로 연결되지 않는다.

- 상담사에게 이관될 때 같은 내용을 여러 번 설명해야 하거나 불필요한 질문을 반복해서 받는다.

기대하는 가치:

- 내 상황에 맞는 안내를 쉽고 빠르게 받아 문제를 빨리 해결하고 싶다.
- 자동 처리로 해결되지 않는 경우에는 적절한 담당자에게 자연스럽게 연결되기를 바란다.

3.2 사용자 여정 (User Journey)

1) 고객 User Journey

1. 고객이 주문·결제·환불·계정 관련 문제를 인지
2. 채팅창에 문의 입력

Happy path

1. 챗봇에게 문제를 해결할 수 있는 답변을 받음
2. 챗봇에게 안내된 방법으로 문제 해결
3. 채팅 세션 종료

Exceptional path

1. 챗봇이 처리 불가능하다는 것과 문의 사항이 담당자에게 이관되었다는 메세지를 받음
2. 채팅 종료
3. 추후 서비스 담당자가 문의사항을 처리했다는 알림을 받음
4. 문제 해결 및 채팅 세션 종료

Failure path

1. 질문에 대해 챗봇이 엉뚱한 답변을 함
2. 상담사에게 문제 이관
3. 상담사와 소통하며 문제 해결

2) 운영 관리자 User Journey

서비스가 문의 유형, 처리 흐름, 예외 대응 기준 초안을 생성한 내용을 확인

Happy path

1. 운영 관리자가 초안을 검토·수정하고 운영 정책으로 확정함
2. 확정된 정책이 상담/챗봇 흐름에 반영
3. 반복 문의는 자동 처리되고 상담사는 복잡한 문의에 집중
4. 운영 효율과 응대 품질을 개선

Exceptional path

1. 서비스가 일부 문의 유형이나 정책을 불확실한 초안으로 생성
2. 운영 관리자가 해당 항목만 직접 수정하거나 보완
3. 수정된 정책을 승인하여 시스템에 반영
4. 반복 문의는 자동 처리되고 상담사는 복잡한 문의에 집중
5. 운영 효율과 응대 품질을 개선

3) 현업 정책 담당자

상담사 또는 챗봇이 처리하지 못한 문의를 이관받음

고객 상황, 문의 맥락, 상담 내역, 관련 정책 정보를 검토함

Happy path

1. 이관된 문의에 필요한 정보와 고객 맥락이 한 번에 정리되어 전달됨
2. 관련 정책, 유사 사례, 권장 처리 방향을 참고해 빠르게 판단
3. 정책에 따라 승인·반려·예외 처리 등 필요한 조치를 수행
4. 처리 결과를 고객 또는 상담사에게 전달
5. 문의가 효율적으로 종결됨

Exceptional path

1. 이관된 문의는 받았지만 정보가 일부 부족하거나 예외 여부 판단이 애매함
2. 정책 담당자가 추가 정보나 확인이 필요한 사항을 다시 요청하거나 찾아봄
3. 보완된 정보를 바탕으로 최종 판단 후 처리
4. 다소 시간이 걸리지만 정책 기준에 맞게 문의를 마무리

3.3 Epics & User Stories

Epic 1: 고객 문의 접수 및 1차 해결

고객이 주문·결제·환불·계정 관련 문제를 인지하고 채팅창에 문의를 입력한 뒤, 챗봇을 통해 1차적으로 문제를 해결하는 과정
(고객 Journey: 문제 인지, 문의 입력, Happy path 전반)

- User Story 1.1: 나는 고객으로서, 주문·결제·환불·계정 관련 문제를 채팅창에 쉽게 입력하고 싶다 → 빠르게 도움을 받기 위해
- User Story 1.2: 나는 고객으로서, 내 문의에 대해 챗봇이 해결 가능한 답변을 제공해주길 원한다 → 상담사 연결 없이 문제를 해결하기 위해
- User Story 1.3: 나는 고객으로서, 챗봇이 안내한 방법을 이해하기 쉽게 받고 싶다 → 스스로 문제를 해결할 수 있기 위해
- User Story 1.4: 나는 고객으로서, 문제 해결 후 채팅 세션을 자연스럽게 종료하고 싶다 → 불필요한 절차 없이 상담을 마무리하기 위해

Epic 2: 처리 불가 및 오응답 문의의 상담 이관

챗봇이 직접 처리할 수 없거나 부정확한 답변을 한 경우, 고객 문의를 적절한 담당자나 상담사에게 이관하는 과정
(고객 Journey: Exceptional path, Failure path 전반)

- User Story 2.1: 나는 고객으로서, 챗봇이 처리할 수 없는 경우 이관 사실을 명확히 안내받고 싶다 → 현재 문의 상태를 알기 위해
- User Story 2.2: 나는 고객으로서, 챗봇의 답변이 적절하지 않을 때 상담사에게 연결되고 싶다 → 잘못된 답변으로 시간을 낭비하지 않기 위해
- User Story 2.3: 나는 고객으로서, 기존 대화 내용이 담당자나 상담사에게 함께 전달되길 원한다 → 같은 내용을 반복 설명하지 않기 위해
- User Story 2.4: 나는 고객으로서, 문의 처리 결과를 추후 알림으로 받고 싶다 → 채팅 종료 후에도 결과를 확인하기 위해
- User Story 2.5: 나는 고객으로서, 담당자 또는 상담사와의 후속 처리로 문제를 최종 해결하고 싶다 → 문의를 끝까지 해결하기 위해

Epic 3: 운영 정책 초안 생성 및 반영

서비스가 문의 유형, 처리 흐름, 예외 대응 기준 초안을 생성하고, 운영 관리자가 이를 검토·수정·확정하여 실제 상담 흐름에 반영하는 과정
(운영 관리자 Journey: Happy path, Exceptional path 전반)

- User Story 3.1: 나는 운영 관리자로서, 서비스가 생성한 문의 유형과 처리 흐름 초안을 한눈에 확인하고 싶다 → 운영 설계를 빠르게 검토하기 위해
- User Story 3.2: 나는 운영 관리자로서, 초안을 검토하고 필요한 부분을 수정·보완하고 싶다 → 실제 운영 정책에 맞게 정교화하기 위해
- User Story 3.3: 나는 운영 관리자로서, 확정된 정책을 상담 및 챗봇 흐름에 반영하고 싶다 → 운영 기준을 실제 서비스에 적용하기 위해
- User Story 3.4: 나는 운영 관리자로서, 불확실하게 생성된 항목만 선택적으로 수정하고 싶다 → 전체를 다시 설계하지 않고도 빠르게 품질을 높이기 위해
- User Story 3.5: 나는 운영 관리자로서, 반복 문의는 자동 처리되고 상담사는 복잡한 문의에 집중할 수 있기를 바란다 → 운영 효율과 응대 품질을 개선하기 위해

Epic 4: 현업 정책 담당자의 이관 문의 검토 및 처리

상담사 또는 챗봇이 처리하지 못한 문의를 현업 정책 담당자가 넘겨받아, 고객 상황과 관련 정보를 검토한 뒤 정책에 따라 판단하고 조치하는 과정
(현업 정책 담당자 Journey: Happy path 전반)

- User Story 4.1: 나는 현업 정책 담당자로서, 이관된 문의의 고객 상황과 상담 맥락을 한 번에 전달받고 싶다 → 빠르게 상황을 이해하기 위해
- User Story 4.2: 나는 현업 정책 담당자로서, 관련 정책, 유사 사례, 권장 처리 방향을 함께 확인하고 싶다 → 정확하고 일관된 판단을 하기 위해
- User Story 4.3: 나는 현업 정책 담당자로서, 정책에 따라 승인·반려·예외 처리 등의 조치를 수행하고 싶다 → 문의를 적절히 처리하기 위해
- User Story 4.4: 나는 현업 정책 담당자로서, 처리 결과를 고객 또는 상담사에게 전달하고 싶다 → 문의를 명확히 종결하기 위해

Epic 5: 예외 문의의 추가 확인 및 안정적 종결

이관된 문의에 정보가 부족하거나 판단이 애매한 경우, 추가 확인과 보완을 거쳐 최종적으로 문제를 해결하는 과정
(현업 정책 담당자 Journey: Exceptional path 전반)

- User Story 5.1: 나는 현업 정책 담당자로서, 이관된 문의에 필요한 정보가 부족한지 식별하고 싶다 → 불완전한 정보로 잘못 판단하지 않기 위해

- User Story 5.2: 나는 현업 정책 담당자로서, 추가 정보나 확인이 필요한 사항을 다시 요청하거나 직접 찾아보고 싶다 → 정확한 판단 근거를 확보하기 위해
- User Story 5.3: 나는 현업 정책 담당자로서, 보완된 정보를 바탕으로 최종 판단을 내리고 싶다 → 정책 기준에 맞게 문의를 처리하기 위해
- User Story 5.4: 나는 현업 정책 담당자로서, 시간이 더 걸리더라도 문의를 안정적으로 마무리하고 싶다 → 예외 상황에서도 처리 품질을 유지하기 위해

4. 설계 및 구현

4.1 기술 분석

4.1.1 기술 스택

- Frontend: React + TypeScript + Vite
    - 운영자 화면, 검토 UI, 데모 채팅 UI 구현에 사용
    - 컴포넌트 기반 개발이 용이하고, TypeScript를 통해 상태와 API 스키마를 안정적으로 관리할 수 있어 선택
    - Vite는 개발 서버 속도와 빌드 속도가 빨라 빠른 반복 개발에 적합
- Backend: Spring Boot + Java
    - 메인 API 서버, 도메인 팩 관리, 검토/승인 로직, 채팅 세션 처리에 사용
    - 제품 서비스의 중심 로직을 안정적으로 구현하기에 적합하며, 구조화된 서버 개발과 유지보수 측면에서 유리하여 선택
- Workflow Orchestration: Airflow
    - 상담 로그 기반 도메인 팩 생성 파이프라인을 단계별로 실행·관리하는 데 사용
    - 특히 Python 기반 ML 코드와 연결하기 쉽고, 전처리부터 초안 생성까지의 오프라인 배치 워크플로우를 관리하기에 적합하여 선택
- 실시간 통신: WebSocket
    - 실시간 채팅 및 상태 업데이트 반영에 사용
    - 요청-응답 방식보다 대화형 인터페이스 구현에 적합하여 선택
- Database: PostgreSQL
    - 서비스 메타데이터, 도메인 팩 버전, 파이프라인 실행 상태, 분석 결과 저장에 사용
    - MySQL 대비 분석성 쿼리와 확장성 측면에서 유리하다고 판단하여 선택
- LLM 연동: OpenRouter / OpenAI API / Bedrock
    - 요약, 정규화, 구조화, 라벨링 등 일부 AI 처리 단계에 활용 예정
    - 특정 공급자에 종속되지 않도록 여러 후보를 검토하며, 실험 단계와 운영 단계를 분리해 유연하게 적용할 계획

4.1.2 개발 및 협업 환경

Infra: AWS ECS(Fargate)

- 컨테이너 기반으로 서비스 배포 및 운영
- 서버 관리 부담을 줄이고, 배포 단위를 명확히 가져가기 위해 선택
- 제품 서비스는 Spring Boot + Frontend(Vite build 결과)를 하나의 축으로 배포하고, Airflow는 별도 파이프라인 축으로 운영

Spring ↔ Airflow 연동

- Spring Boot가 Airflow에 API 호출로 파이프라인 실행을 요청하고, 작업 완료 후 Airflow가 웹훅으로 결과를 전달하는 방식으로 구성
- 이를 통해 제품 서비스와 오프라인 생성 파이프라인의 책임을 분리

CI/CD: GitHub Actions

- 빌드, 테스트, 배포 자동화에 활용
- GitHub 중심 협업 흐름과 자연스럽게 연결되며, 반복 작업을 줄일 수 있어 선택

개발 전략: Monorepo + Trunk-based Development + Squash Merge

- 긴 브랜치 운영을 피하고 빠르게 통합하기 위한 방식
- 커밋 히스토리를 단순하게 유지하고 협업 충돌을 줄이기 위해 적용

코드 리뷰: Pull Request + CodeRabbit

- Pull Request 기반 리뷰 프로세스를 기본으로 하고, CodeRabbit을 활용해 자동 리뷰를 보조
- 리뷰 품질을 높이고 반복적인 실수를 줄이기 위해 적용

4.1.3 핵심 기술 및 알고리즘

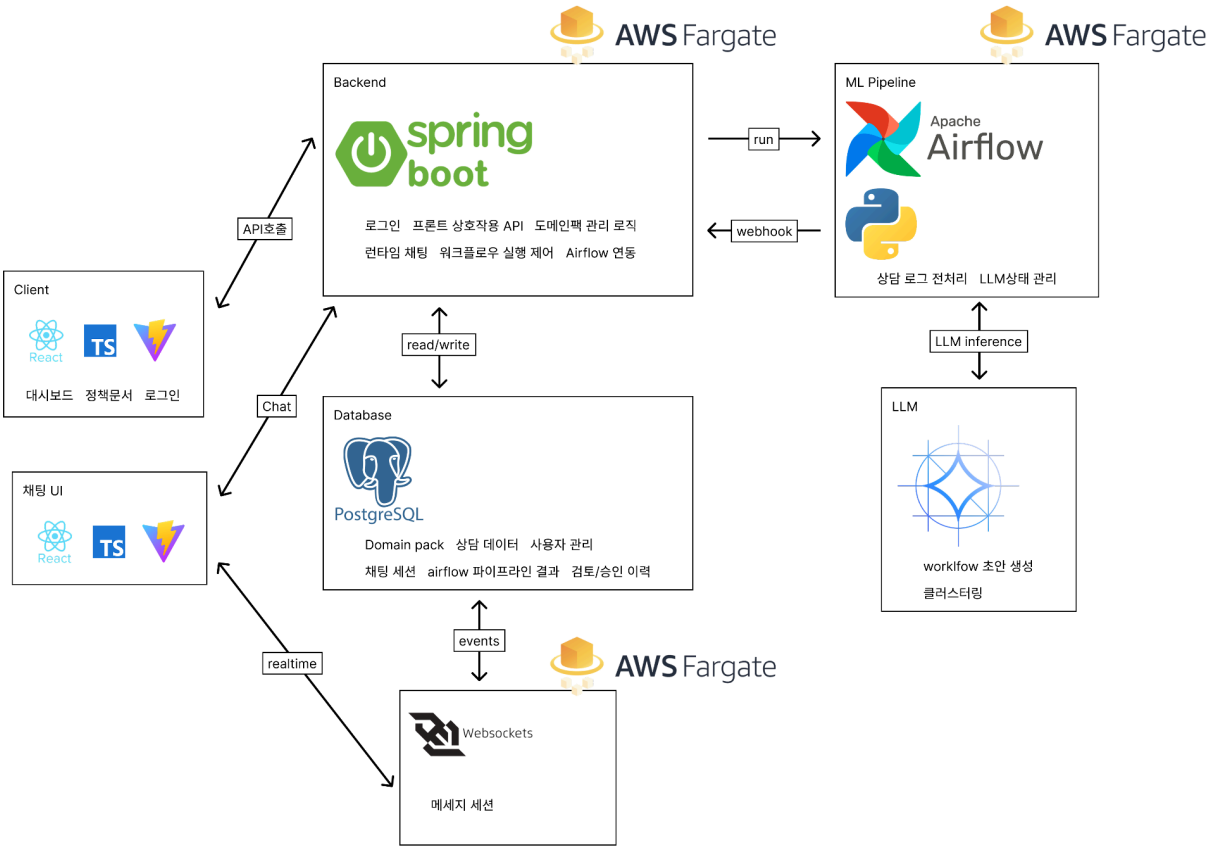
- 상담 로그를 conversation 단위로 정제하고, 화자 역할을 반영한 role-aware representation을 구성한다.
- semantic embedding, 그래프 커뮤니티 탐지, flow signature를 결합해 workflow entry point가 될 수 있는 intent 후보를 도출한다.
- 도출된 intent 후보를 기반으로 slot, policy, risk, workflow 초안을 생성한다.
- 생성된 초안은 사람 검토 루프를 통해 보정·승인하고, 최종적으로 domain pack 형태로 관리한다.

4.2 시스템 설계

4.2.1 시스템 개요

우리 시스템은 상담 로그를 분석해 domain pack 초안을 생성하고, 이를 사람 검토와 버전 관리를 거쳐 실제 채팅 워크플로우에 반영하는 AI 기반 고객 상담 운영 시스템이다.
운영자 콘솔과 채팅 UI를 통해 들어온 요청을 Spring Boot 백엔드가 처리하고, PostgreSQL에 서비스 상태와 데이터를 저장하며, Airflow 기반 ML 파이프라인을 통해 상담 로그 분석 및 workflow 초안 생성을 수행하는 구조로 이루어진다.
Spring Boot는 인증, 도메인팩 관리, 검토 이력 관리, 채팅 실행 제어 등 서비스의 중심 비즈니스 로직을 담당한다. PostgreSQL은 사용자, 세션, 상담 데이터, domain pack, 파이프라인 결과, 검토·승인 이력 등 주요 데이터를 저장하는 기준 저장소 역할을 한다. Airflow는 상담 로그 전처리, 생성 작업 orchestration, LLM 호출 기반 초안 생성 파이프라인을 실행하며, 작업 결과는 webhook을 통해 백엔드에 반영된다.
채팅 런타임에서는 채팅 UI의 요청이 Spring Boot로 전달되어 처리되고, 처리 결과 및 상태 변화는 WebSocket을 통해 실시간으로 UI에 전달된다. 이를 통해 우리 시스템은 운영 관리 기능, 생성 파이프라인, 실시간 채팅 기능이 통합된 형태로 동작한다.

4.2.2 소프트웨어 아키텍처



전체 시스템은 제품 런타임 계층과 도메인 팩 생성 계층으로 분리하여 설계한다.
제품 런타임 계층은 운영자 UI, 검토·승인, 채팅 데모, domain pack 조회·적용과 같이 사용자 요청에 실시간으로 반응하는 서비스 기능을 담당한다. 반면 도메인 팩 생성 계층은 상담 로그 전처리, intent discovery, slot/policy/risk/workflow 초안 생성과 같이 대량 데이터를 기반으로 domain pack 초안을 생성하는 오프라인 파이프라인을 담당한다.
이를 통해 실시간 서비스 처리와 장시간 실행되는 생성 작업의 책임을 분리하고, 각 계층을 목적에 맞게 독립적으로 설계·운영할 수 있도록 한다.

Spring Backend

시스템의 백엔드는 DDD(Domain-Driven Design) 기반 모듈형 모놀리스 구조의 Spring Boot 애플리케이션으로 설계한다. 즉, 전체 서비스는 하나의 Spring Boot 애플리케이션으로 통합 운영하되, 내부 구조는 업무 도메인 기준으로 명확히 분리된 모듈 형태로 구성한다.

먼저, DDD는 복잡한 서비스를 기술 계층이 아니라 실제 업무 개념과 흐름을 기준으로 설계하는 방식이다. 즉, 사용자, 주문, 검토, 정책, 실행 흐름처럼 서비스 안의 핵심 도메인을 중심으로 책임과 구조를 나누어, 코드가 실제 비즈니스 구조를 잘 반영하도록 만드는 설계 접근법이다. 우리 서비스는 사용자/권한 관리, 상담 로그 관리, Domain Pack 관리, 검토·승인, 파이프라인 연동, 채팅 런타임 등 다양한 기능을 포함하고 있다. 이러한 기능은 단순한 기술 계층 기준으로 나누기보다, 실제 업무 개념과 흐름을 기준으로 나누는 것이 더 적절하다. 예를 들어 상담 로그로부터 생성된 초안은 검토·승인 과정을 거쳐 Domain Pack으로 관리되고, 다시 채팅 런타임에서 참조되어 실제 응답 흐름에 반영된다. 이처럼 각 기능은 독립적인 화면이나 API로 존재하더라도 실제로는 하나의 업무 흐름 안에서 긴밀하게 연결되어 있다. 따라서 DDD를 적용하면 코드 구조가 서비스의 실제 구조를 잘 반영할 수 있고, 복잡한 비즈니스 로직도 도메인 중심으로 명확하게 정리할 수 있다.

다음으로, 모듈형 구조는 하나의 애플리케이션 안에서 기능이나 도메인별로 코드를 독립된 단위로 나누어 구성하는 방식이다. 각 모듈은 자신의 책임을 중심으로 묶여 있고 모듈 간 의존성을 통제하여, 기능이 많아져도 구조를 명확하게 유지하고 변경 영향을 줄이기 쉽게 만들기 때문에 도메인 간 책임과 변경 범위를 통제하는 데 적합하다. 우리 서비스는 app, corpus, pack, review, pipeline, runtime과 같이 성격이 다른 업무 영역으로 구분할 수 있다. 이를 하나의 프로젝트 안에서 독립된 모듈로 구성하면 각 도메인의 책임을 분명히 할 수 있고, 특정 기능 수정이 다른 영역으로 불필요하게 확산되는 것을 줄일 수 있다. 예를 들어 review 영역의 변경이 runtime이나 pipeline 영역에 직접적인 영향을 주지 않도록 경계를 설정할 수 있으며, 이를 통해 코드 응집도를 높이고 유지보수성을 향상시킬 수 있다. 즉, 모듈형 구조는 기능이 많고 도메인 간 관계가 복잡한 우리 서비스에서 내부 구조를 질서 있게 유지하는 데 유리하다.

또한, 모놀리스 구조는 현재 단계의 서비스 운영 방식에 적합하다. 우리 서비스는 다양한 서브도메인을 포함하지만, 이들이 완전히 독립적으로 운영되기보다는 PostgreSQL, Airflow, LLM, WebSocket과 연계되어 하나의 서비스 맥락 안에서 함께 동작한다. Spring Backend는 운영 기능, 생성 파이프라인, 실시간 채팅 기능을 중앙에서 조율하며, 데이터 정합성과 상태 전이가 중요한 흐름을 다룬다. 이러한 상황에서 초기부터 마이크로서비스로 과도하게 분리할 경우, 서비스 간 통신 비용, 배포 복잡성, 장애 추적 난이도, 데이터 정합성 관리 부담이 크게 증가할 수 있다. 반면 모놀리스 구조는 단일 애플리케이션 기반의 간결한 배포 구조와 일관된 트랜잭션 처리를 유지할 수 있어, 현재 단계에서 더 현실적이고 안정적인 선택이다. 또한 로컬 개발, 테스트, 운영 측면에서도 복잡한 분산 환경 없이 빠른 개발과 검증이 가능하다.

따라서 우리시스템은 DDD를 통해 도메인 중심의 업무 구조를 반영하고, 모듈형 구조를 통해 내부 책임과 의존성을 통제하며, 모놀리스 구조를 통해 운영 단순성과 트랜잭션 일관성을 확보하는 방향이 가장 적합하기 때문에 DDD 기반 모듈형 모놀리스 구조의 Spring Boot 백엔드를

채택한다. 복잡한 도메인 구조를 효과적으로 관리하면서도 현재 단계에 필요한 개발 효율성과 운영 안정성을 함께 확보하고자 한다. 또한 이 구조는 향후 특정 기능의 트래픽 증가나 독립 배포 필요성이 커질 경우, 개별 모듈을 별도 서비스로 두어 MSA로 확장할 수 있는 기반도 제공한다.

bounded context

Bounded Context는 특정 업무 영역의 개념, 규칙, 책임이 일관되게 적용되는 경계를 의미한다. 즉, 서로 다른 기능을 단순히 기술 계층으로 나누는 것이 아니라, 서비스 안에서 독립적인 의미와 역할을 가지는 업무 단위별로 경계를 설정하는 방식이다. 우리 시스템은 이러한 bounded context 기준으로 백엔드 내부를 모듈화하여, 각 영역의 책임을 명확히 분리하고 변경 영향 범위를 최소화하도록 설계한다. 이에 따라 우리 시스템의 모듈형 구조는 다음과 같이 구성한다.

bounded context: domain-pack

- 역할
  - intent / slot / policy / risk / workflow 초안과 확정본을 버전 단위로 관리
  - publish 상태와 배포 가능한 domain pack 버전을 제어
- 주요 Aggregate
  - DomainPack
- 주요 내부 요소
  - DomainPackVersion
  - IntentDefinition
  - SlotDefinition
  - PolicyDefinition
  - RiskDefinition
  - WorkflowDefinition

bounded context: review

- 역할
  - 생성된 초안에 대한 검토, 수정, 승인/반려, 코멘트 이력을 관리
  - 사람 검토 루프를 통해 domain pack 품질을 보정
- 주요 Aggregate
  - ReviewSession
- 주요 내부 요소
  - ReviewTask
  - ReviewComment

    - ReviewDecision
    - ReviewStatus

bounded context: pipeline-job

- 역할
    - Airflow 파이프라인 실행 요청
    - job 상태 추적
    - 웹훅 결과 수신 및 domain-pack 반영
- 주요 Aggregate
    - PipelineJob
- 주요 내부 요소
    - PipelineRunRef
    - JobStatus
    - PipelineResultRef
    - WebhookReceipt

bounded context: workflow-runtime

- 역할
    - publish된 domain pack을 읽어 현재 대화 상태를 해석
    - 다음 action을 결정하고 answer / handoff 흐름을 제어
- 주요 Aggregate
    - WorkflowExecution
- 주요 내부 요소
    - ConversationState
    - SlotState
    - PolicyCheckResult
    - RiskCheckResult
    - NextAction

bounded context: chat

- 역할
    - 데모용 채팅 세션, 메시지 기록, WebSocket 이벤트 관리
    - workflow-runtime 결과를 사용자 화면에 반영
- 주요 Aggregate
    - ChatSession

- 주요 내부 요소
    - ChatMessage
    - Participant
    - SessionStatus

shared / infra

- 역할
    - 인증, 예외 처리, 공통 응답, DB 설정, 외부 연동 클라이언트 제공
- 주요 요소
    - 공통 예외 처리기
    - 공통 응답 포맷
    - JPA 설정
    - WebSocket 설정
    - 공통 이벤트 처리

Airflow + Python Pipeline

우리 시스템은 파이프라인 오케스트레이터로 Airflow, 데이터 처리 및 ML 연산을 위해 Python을 사용한다.
대량의 상담 데이터를 순차적으로 정제하고 분석하여 산출물을 만들어내는 배치성 작업을 처리하기 위해 Airflow를 사용한다. 실제 전처리, 임베딩, 클러스터링, 초안 생성, 평가 로직은 데이터 처리 및 ML 연산에 적합한 Python 기반으로 구현하여, NLP/ML 라이브러리와의 연계성과 실험 유연성을 확보한다.

주요 파이프라인 모듈

- ingestion
    - 상담 로그 입력
    - conversation 단위 묶기
    - role 정리 및 비식별화
- preprocessing
    - boilerplate 제거
    - canonical text 생성
    - role-aware representation 생성
- intent-discovery
    - semantic embedding
    - graph community detection

    - flow signature 생성
    - hybrid clustering 수행
- draft-generation
    - intent별 slot / policy / risk / workflow 초안 생성
    - intent card 또는 draft artifact 형태로 산출물 생성
- evaluation
    - mapping rate
    - outlier rate
    - workflow separability
    - merge/split effort 측정
- publish-candidate
    - 검토 가능한 draft artifact 생성
    - Spring Backend로 결과 전달

적용 패턴

- DAG 기반 Task Orchestration : 작업 간 선후관계와 의존성을 그래프로 관리하여 파이프라인을 순서대로 안정적인 실행순서와 의존성의 명확한 관리를 위해 사용
- Artifact-driven Processing : 각 단계의 결과를 독립된 산출물로 관리하는 방식으로, 중간 결과를 추적·검증·재사용을 위해 사용
- Stage-based Pipeline : 전체 처리 흐름을 ingestion, preprocessing, generation 같은 단계별로 나누는 방식으로, 구조를 명확하게 하고 유지보수를 쉽게 하기 위해 사용

Spring ↔ Airflow 연동

Spring Boot 백엔드는 Airflow에 API 호출 방식으로 파이프라인 실행을 요청하고, Airflow는 작업 완료 후 웹훅을 통해 실행 결과를 백엔드에 전달한다.
이러한 연동 구조를 통해 사용자 요청을 처리하는 온라인 서비스 계층과, 시간이 오래 걸리는 생성·분석 파이프라인 계층의 책임을 분리한다. 백엔드는 파이프라인 작업 생성과 실행 상태 관리의 기준점 역할을 하며, Airflow로부터 도메인 팩 초안 결과를 수신한 뒤 이를 검토 대상에 등록하는 흐름으로 연계된다.

Frontend

프론트엔드는 운영자 콘솔과 채팅 데모 UI로 구성되며, 주요 화면 모듈은 Domain Pack 목록 및 상세 조회, Review 작업 화면, Pipeline 실행 및 상태 확인 화면, Chat Demo 화면으로 구성한다.

주요 화면 모듈은 아래와 같이 구성한다.

- domain pack 목록 / 상세
- review 작업 화면
- pipeline 실행 / 상태 화면
- chat demo 화면

실시간 상태 변경과 채팅 이벤트는 사용자 화면에 즉시 반영되어야 하므로 WebSocket을 적용한다.

또한 프론트엔드는 Domain Pack, Review, Pipeline, Chat과 같이 성격이 다른 기능을 포함하므로, 기능 단위 모듈 분리 구조를 적용해 화면 책임을 명확히 하고 유지보수성과 확장성을 높인다.

Infra : Fargate

Spring Boot, WebSocket, Airflow 같은 컨테이너 기반 서비스를 운영하기 위해 AWS Fargate를 사용한다.
Spring Boot와 WebSocket은 계속 떠 있어야 하는 장기 실행 서비스라서 Fargate의 대표적인 사용처와 잘 맞고, CPU·메모리를 서비스별로 명확히 나눠 줄 수 있어 백엔드 API 서버, 실시간 통신 서버를 독립적으로 운영하기 좋다. 그리고 서버 패치, 용량 계획, 클러스터 관리 부담을 줄여 팀이 인프라보다 서비스 개발에 집중하기 쉽다.

LLM & AI

Clustering

우리 시스템은 flow feature 기반 클러스터링을 주요 군집화 방식으로 채택한다.
상담 로그로부터 domain pack 초안을 생성하기 위해 다양한 클러스터링 실험을 비교하였다.
실험 결과, 단순 텍스트 기반 군집화나 LLM 요약 기반 표현보다 상담의 진행 구조와 업무 흐름을 직접 반영하는 flow feature 기반 방식이 더 우수한 성능을 보였다. 특히 exp08은 transition feature를 활용한 flow 기반 클러스터링에서 전체 실험 중 가장 높은 silhouette score 0.9075를 기록하였고, 상담 흐름의 반복 패턴을 가장 잘 분리하는 결과를 보였다. 이는 표면적인 문장 유사성보다 실제 상담 절차와 상호작용 순서가 intent 구분에 더 강한 신호가 된다는 점을 보여준다.
이에 따라 우리 시스템은 exp08의 flow feature 기반 클러스터링을 주요 군집화 방식으로 채택하고, 그 결과를 기반으로 LLM이 각 군집의 의미를 해석하여 intent, slot, policy, risk, workflow 초안을 생성하는 구조를 적용한다. 즉, 클러스터링 단계에서는 상담 로그를 구조적으로 분리하고, LLM 단계에서는 각 군집의 representative flow sequence와 exemplar conversation을 바탕으로 사람이 검토 가능한 domain pack 초안을 생성한다. 이를 통해 군집 형성은 데이터 기반으로 안정적으로 수행하고, 초안 생성은 LLM을 통해 구조화된 산출물로

확장할 수 있다.

Workflow 초안 생성

우리 시스템에서 Workflow는 단순한 지시사항 텍스트가 아니라, 상담 처리 절차를 상태(state)와 분기 조건(transition)으로 표현한 상태 기반 처리 흐름으로 정의한다. 즉, 특정 문의 유형에 대해 어떤 정보를 추가로 수집해야 하는지, 어떤 운영 정책을 확인해야 하는지, 어떤 위험 조건에서 자동 처리를 중단하고 상담사에게 이관해야 하는지, 그리고 최종적으로 어떤 순서로 처리해야 하는지를 구조적으로 표현하는 실행 단위이다.

LLM은 이러한 Workflow를 직접 실행하는 주체가 아니라, 상담 로그와 대표 상담 흐름을 바탕으로 intent, slot, policy, risk, workflow 초안을 생성하는 역할을 수행한다. 이때 workflow 초안은 “다음에 어떤 문장을 답할지”를 생성하는 형태가 아니라, “어떤 상태에서 무엇을 확인하고, 어떤 조건이면 다음 단계로 이동하거나 이관할지”를 정의한 graph 형태의 초안으로 생성된다.

이후 Spring Backend와 runtime 계층은 이 초안을 해석하여 실제 채팅 실행 시 다음 질문, 자동 응답, 정책 확인, 상담사 이관 등의 동작을 결정한다.

4.2.3 데이터 설계

전체 DB 구조

- app: 워크스페이스 및 사용자 관리
- corpus: 상담 로그 원천 및 전처리 데이터
- pack: domain pack 및 intent / slot / policy / risk / workflow 정의
- review: 사람 검토 및 승인 이력
- pipeline: Airflow job, artifact, 평가 메트릭
- runtime: 채팅 데모, workflow 실행 상태, decision log

schema: app

- workspace
- app_user
- workspace_member
- 역할: 시스템의 최상위 작업 단위와 사용자/권한 관리

schema: corpus

- dataset
- conversation

- conversation_turn
- conversation_feature_snapshot
- 역할: 상담 로그 원본, turn 단위 대화, canonical text / flow signature 같은 전처리 결과 저장

schema: pack

- domain_pack
- domain_pack_version
- intent_definition
- slot_definition
- intent_slot_binding
- policy_definition
- risk_definition
- workflow_definition
- intent_workflow_binding
- 역할: 핵심 산출물인 domain pack과 그 하위 정의를 버전 단위로 관리

schema: review

- review_session
- review_task
- review_comment
- review_decision
- review_activity_log
- review_summary_metric
- 역할: AI가 만든 초안을 사람이 검토·수정·승인하는 human-in-the-loop 구조 지원

schema: pipeline

- pipeline_job
- pipeline_job_event
- pipeline_artifact
- webhook_receipt
- evaluation_run
- evaluation_metric
- cluster_evaluation
- novel_intent_candidate
- taxonomy_drift_log

- 역할: Airflow 파이프라인 실행 상태, 산출물, 평가 지표, 신규 intent/drift 추적

schema: runtime

- chat_session
- chat_message
- workflow_execution
- workflow_execution_step
- decision_log
- session_outcome
- 역할: publish된 domain pack 기반의 채팅 데모 실행, 상태 전이, 자동응답/이관 결정 근거 기록

핵심 관계 구조

- workspace → domain_pack → domain_pack_version
- domain_pack_version → intent / slot / policy / risk / workflow
- domain_pack_version → review_session → review_task
- dataset → pipeline_job → pipeline_artifact / evaluation_metric
- chat_session → workflow_execution → decision_log
- 즉, 상담 로그(corpus) → 생성 파이프라인(pipeline) → 도메인 팩(pack) → 검토(review) → 실행(runtime) 흐름으로 연결된다.

4.2.4 API 정의

우리 시스템의 외부 API는 /api/v1 prefix를 기준으로 구성한다.
주요 리소스는 Domain Pack, 정의 항목(Intent / Slot / Policy / Risk / Workflow), Review, Pipeline Job, Dataset, Chat Runtime으로 구분한다.
관리 기능과 조회·수정 작업은 REST API로 제공하고, 채팅 데모의 실시간 상태 반영은 WebSocket으로 처리한다.
또한 Spring Backend와 Airflow 간의 장시간 실행 작업 연동은 내부 API와 Webhook 방식으로 분리한다.

1. REST API

1.1 Domain Pack

Domain Pack과 버전 단위의 생성, 조회, 수정, 상태 전이를 처리하는 API이다.

- GET /api/v1/domain-packs

- POST /api/v1/domain-packs
- GET /api/v1/domain-packs/{packId}
- PATCH /api/v1/domain-packs/{packId}
- GET /api/v1/domain-packs/{packId}/versions
- POST /api/v1/domain-packs/{packId}/versions
- GET /api/v1/domain-pack-versions/{versionId}
- PATCH /api/v1/domain-pack-versions/{versionId}
- POST /api/v1/domain-pack-versions/{versionId}/submit-review
- POST /api/v1/domain-pack-versions/{versionId}/publish
- POST /api/v1/domain-pack-versions/{versionId}/archive

1.2 Intent / Slot / Policy / Risk / Workflow Definition

각 Domain Pack Version에 속한 정의 항목을 관리하는 API이다.
Intent, Slot, Policy, Risk, Workflow는 동일한 구조로 조회·생성·수정 API를 제공한다.

Intent

- GET /api/v1/domain-pack-versions/{versionId}/intents
- POST /api/v1/domain-pack-versions/{versionId}/intents
- GET /api/v1/intents/{intentId}
- PATCH /api/v1/intents/{intentId}

Slot

- GET /api/v1/domain-pack-versions/{versionId}/slots
- POST /api/v1/domain-pack-versions/{versionId}/slots
- GET /api/v1/slots/{slotId}
- PATCH /api/v1/slots/{slotId}

Policy

- GET /api/v1/domain-pack-versions/{versionId}/policies
- POST /api/v1/domain-pack-versions/{versionId}/policies
- GET /api/v1/policies/{policyId}
- PATCH /api/v1/policies/{policyId}

Risk

- GET /api/v1/domain-pack-versions/{versionId}/risks
- POST /api/v1/domain-pack-versions/{versionId}/risks
- GET /api/v1/risks/{riskId}
- PATCH /api/v1/risks/{riskId}

Workflow

- GET /api/v1/domain-pack-versions/{versionId}/workflows
- POST /api/v1/domain-pack-versions/{versionId}/workflows
- GET /api/v1/workflows/{workflowId}
- PATCH /api/v1/workflows/{workflowId}

1.3 Review

AI가 생성한 초안에 대해 검토 세션과 검토 작업을 생성하고, 승인·반려·수정 요청 등의 의사결정을 처리하는 API이다.

Review Session

- GET /api/v1/review-sessions
- POST /api/v1/review-sessions
- GET /api/v1/review-sessions/{reviewSessionId}
- PATCH /api/v1/review-sessions/{reviewSessionId}
- POST /api/v1/review-sessions/{reviewSessionId}/close

Review Task

- GET /api/v1/review-sessions/{reviewSessionId}/tasks
- POST /api/v1/review-sessions/{reviewSessionId}/tasks
- GET /api/v1/review-tasks/{taskId}
- PATCH /api/v1/review-tasks/{taskId}
- POST /api/v1/review-tasks/{taskId}/comments
- POST /api/v1/review-tasks/{taskId}/approve
- POST /api/v1/review-tasks/{taskId}/reject
- POST /api/v1/review-tasks/{taskId}/request-change
- POST /api/v1/review-tasks/{taskId}/merge
- POST /api/v1/review-tasks/{taskId}/split

1.4 Pipeline Job

상담 로그 기반 생성 파이프라인의 실행 요청, 상태 조회, 산출물 확인을 위한 API이다.

- GET /api/v1/pipeline-jobs
- POST /api/v1/pipeline-jobs
- GET /api/v1/pipeline-jobs/{jobId}
- POST /api/v1/pipeline-jobs/{jobId}/cancel
- GET /api/v1/pipeline-jobs/{jobId}/events
- GET /api/v1/pipeline-jobs/{jobId}/artifacts
- GET /api/v1/pipeline-jobs/{jobId}/evaluations
- GET /api/v1/pipeline-jobs/{jobId}/novel-intent-candidates

1.5 Dataset / Corpus

업로드된 데이터셋과 대화 원문, turn, 전처리 결과를 조회하는 API이다.

- GET /api/v1/datasets
- POST /api/v1/datasets
- GET /api/v1/datasets/{datasetId}
- GET /api/v1/datasets/{datasetId}/conversations
- GET /api/v1/conversations/{conversationId}
- GET /api/v1/conversations/{conversationId}/turns
- GET /api/v1/conversations/{conversationId}/feature-snapshots

1.6 Chat Demo / Runtime

데모용 채팅 세션 생성, 메시지 송수신, workflow 실행 상태와 결과 조회를 위한 API이다.

Chat Session

- POST /api/v1/chat-sessions
- GET /api/v1/chat-sessions/{chatSessionId}
- GET /api/v1/chat-sessions/{chatSessionId}/messages
- POST /api/v1/chat-sessions/{chatSessionId}/messages
- POST /api/v1/chat-sessions/{chatSessionId}/close

Workflow Runtime

- GET /api/v1/chat-sessions/{chatSessionId}/workflow-execution
- GET /api/v1/workflow-executions/{executionId}

- GET /api/v1/workflow-executions/{executionId}/steps
- GET /api/v1/workflow-executions/{executionId}/decision-logs
- GET /api/v1/chat-sessions/{chatSessionId}/outcome

2. WebSocket

WebSocket은 채팅 데모 화면의 실시간 상호작용을 위해서만 사용한다.
즉, Domain Pack, Review, Pipeline Job과 같은 관리 기능은 REST API로 처리하고,
채팅 메시지와 workflow 실행 상태처럼 즉시 화면 반영이 필요한 정보만 WebSocket으로 전달한다.

2.1 연결 경로

- /ws/chat-sessions/{chatSessionId}

2.2 주요 이벤트

chat.message.created

- 고객, assistant, operator 메시지가 생성되었을 때 전달되는 이벤트

workflow.execution.updated

- 현재 workflow execution 상태가 변경되었을 때 전달되는 이벤트

workflow.step.created

- 새로운 상태 전이(step)가 생성되었을 때 전달되는 이벤트

workflow.decision.logged

- intent 판단, slot 부족, policy / risk 체크 등 의사결정 로그가 기록되었을 때 전달되는 이벤트

chat.session.closed

- 채팅 세션이 종료되었을 때 전달되는 이벤트

3. 내부 연동 API / Webhook

Spring Backend와 Airflow는 장시간 실행되는 파이프라인 작업을 분리하기 위해 내부 API와 Webhook 방식으로 연동한다.

3.1 Spring → Airflow

Spring Backend가 Airflow에 파이프라인 실행을 요청하거나 실행 상태를 조회할 때 사용하는 내부 API이다.

- POST /internal/pipeline-jobs
- GET /internal/pipeline-jobs/{externalRunId}

3.2 Airflow → Spring Webhook

Airflow가 파이프라인 완료 또는 상태 변경 결과를 Spring Backend로 전달할 때 사용하는 Webhook이다.

- POST /internal/webhooks/pipeline-jobs/{jobId}

5. 과제 수행 계획

5.1 일정계획

- 번다운 차트로 대체하며 기획제안서에 포함시키지 않음

5.2 업무분장

- 기획 및 요구사항 정리 : 강희원, 배성연
- 아키텍처·설계 문서화 : 모든 팀원
- 발표자료 제작 : 배성연
- 발표 및 데모 시나리오 준비 : 모든 팀원
- 실험 결과 정리 및 성과 분석 : 강희원, 강준현, 하장한
- 프로젝트 일정 및 협업 관리 : 모든 팀원
- 최종 검수 및 제출물 관리 : 배성연

5.3 협업 및 데브옵스 자동화

- 코딩 컨벤션, 코드리뷰 방식, 테스트 자동화 방식 및 도구 설명: 앞의 기술 분석에서 이이 소개된 것은 여기에 중복시킬 필요 없음

5.4 성과 측정 방안

우리 시스템의 성과는 도메인 팩 생성 품질, 검토 효율성, 채팅 실행 품질 관점에서 정량적·정성적으로 측정한다.

1. 정량적 KPI

정량적 KPI는 생성 파이프라인과 운영 결과를 수치로 측정하기 위한 지표로 설정한다.

- 클러스터링 품질 지표
    - silhouette score
    - outlier rate

    - pairwise accuracy
    - workflow separability

      상담 로그가 의미 있는 intent 단위로 얼마나 잘 분리되었는지 평가한다.
- 초안 생성 품질 지표
    - intent / slot / policy / risk / workflow 초안 생성 성공률
    - 검토 반려율
    - 초안 수정률
    - merge / split 발생 비율

      AI가 생성한 초안이 실제 검토 가능한 수준인지 측정한다.
- 검토 효율성 지표
    - review 완료 소요 시간
    - review task당 평균 코멘트 수
    - 승인까지 필요한 수정 횟수

      human-in-the-loop 과정이 얼마나 효율적으로 작동하는지 평가한다.
- 운영 적용 지표
    - publish 가능한 domain pack 생성률
    - 신규 intent 반영률
    - taxonomy drift 탐지 건수

      생성 결과가 실제 운영 가능한 자산으로 얼마나 전환되는지 측정한다.
- 런타임 성능 지표
    - 자동 처리율
    - 상담사 이관율
    - 평균 응답 시간
    - workflow 완료율

      실제 채팅 실행 단계에서 workflow 기반 처리가 얼마나 안정적으로 수행되는지 측정한다.

2. 정성적 KPI

정성적 KPI는 실제 사용자와 운영자가 체감하는 품질을 평가하기 위한 지표로 설정한다.

- 운영자 관점
    - 생성된 domain pack의 해석 가능성
    - review UI의 사용 편의성
    - workflow 초안의 수정 용이성
    - 결과에 대한 신뢰도
- 상담 운영 관점
    - intent 분류 결과의 업무 적합성

    - slot / policy / risk 정의의 실무 활용성
    - workflow 초안이 실제 상담 절차를 잘 반영하는지 여부
- 채팅 사용자 관점
    - 응답 흐름의 자연스러움
    - 문제 해결 가능성
    - 상담사 이관 시점의 적절성

3. 종합 평가 방식

정량 지표를 통해 군집 품질, 초안 품질, 검토 효율성, 런타임 성능을 측정하고, 정성 지표를 통해 실무 활용성과 사용자 체감 품질을 함께 평가한다. 이를 바탕으로 domain pack 생성 파이프라인과 workflow 초안 생성 품질을 지속적으로 개선한다.